SECRET GOALS

Raport z Etapu 2: Architektoniczny setup projektu

Grupa 5C: Łukasz Michno, Aleksandra Nowak
24 października 2026

1. Informacje Ogólne

- **Nazwa grupy projektowej:** 5C
- **Skład zespołu:** Aleksandra Nowak, Łukasz Michno
- **Wybrane IDE:** Visual Studio Code
- **Link do repozytorium GitHub:** https://github.com/lmichno/SecretGoals
- **Zakres etapu:** architektoniczny setup projektu, czyli:
  - wstępnie skonfigurowane repozytorium Git ze zdefiniowaną strukturą folderów zgodną z przyjętym standardem architektury i języka programowania oraz adekwatny plik `.gitignore`,
  - wybrany i zainicjowany framework/biblioteka,
  - standardy kodowania oraz narzędzia do ich automatycznego wymuszania (linter, formatter).

Dodatkowo w tym etapie powstała nowa wersja prototypu interfejsu (v2), która zastępuje wcześniejsze warianty A i B z Etapu 1.

2. Repozytorium Git i struktura folderów

2.1. Przyjęta architektura

Projekt jest zorganizowany jako **monorepo** z dwiema niezależnymi aplikacjami Node.js: frontendem (`client/`) i backendem (`server/`), oraz katalogiem dokumentacji (`docs/`). Każda aplikacja ma własny `package.json`, a plik w katalogu głównym spina je skryptami (uruchamianie obu części jednym poleceniem, wspólny linter i formatter).

Backend ma architekturę warstwową typową dla aplikacji Express: **routes** (definicje endpointów REST) → **controllers** (obsługa żądania i odpowiedzi) → **services** (logika biznesowa) → **Prisma** (dostęp do bazy). Frontend dzieli kod według roli: strony/zakładki, komponenty wielokrotnego użytku, własne hooki i warstwa komunikacji z API.

2.2. Struktura katalogów

```
SecretGoals/
├── client/                    Frontend (React + Vite + Tailwind CSS)
│   ├── public/                pliki statyczne
│   └── src/
│       ├── api/               komunikacja z backendem (fetch /api/...)
│       ├── components/        komponenty UI wielokrotnego użytku
│       ├── hooks/             własne hooki React
│       ├── pages/             zakładki: Cele, Społeczność, Profil
│       ├── App.jsx
│       └── main.jsx
├── server/                    Backend (Node.js + Express)
│   ├── prisma/schema.prisma   schemat bazy danych (SQLite)
│   └── src/
│       ├── controllers/       healthController.js, goalsController.js
│       ├── middleware/        middleware Express (np. autoryzacja)
│       ├── routes/            health.js, goals.js
│       ├── services/          logika biznesowa
│       ├── prisma.js          klient Prisma
│       └── index.js           punkt wejścia serwera
├── docs/                      badanie rynku, user stories, SysML, prototypy, raporty
├── .husky/pre-commit          hook Git uruchamiający lint-staged
├── .editorconfig  .gitattributes  .gitignore
├── .oxlintrc.json  .prettierrc.json  .prettierignore
├── package.json               skrypty: dev, build, lint, format, check
└── README.md
```

2.3. Plik .gitignore i .gitattributes

Wzorzec	Powód wykluczenia z repozytorium
`node_modules/`	zależności odtwarzane poleceniem `npm install` z `package-lock.json`
`dist/`, `dist-ssr/`	wynik budowania frontendu, generowany automatycznie
`.env`, `.env.*` (oprócz `.env.example`)	sekrety i konfiguracja lokalna; wzór zmiennych jest w `server/.env.example`
`*.db`, `*.db-journal`	lokalna baza SQLite, każdy członek zespołu tworzy ją przez `npm run prisma:push`
logi npm/yarn, `.vscode/*`, `.idea`, `.DS_Store`, `Thumbs.db`	pliki tymczasowe, ustawienia edytorów i systemów operacyjnych

Plik `.gitattributes` wymusza końce linii LF dla plików tekstowych, dzięki czemu kod edytowany na Windowsie i Linuksie nie generuje sztucznych różnic w historii. Pliki binarne (PNG, JPG, PDF, bazy) są oznaczone jako `binary`.

3. Wybrany i zainicjowany framework

Warstwa systemu	Technologia i wersja	Stan inicjalizacji
Frontend	React 19, Vite 8, Tailwind CSS 4, Lucide Icons	projekt utworzony z szablonu Vite, Tailwind podpięty przez `@tailwindcss/vite`, proxy `/api` na port 5000
Backend	Node.js 22, Express 4 (moduły ESM)	serwer z CORS i JSON, endpointy `GET /api/health`, `GET /api/goals`, `POST /api/goals`
Baza danych	SQLite + Prisma ORM 6	schemat z modelami `User` (alias) i `Goal`, synchronizacja przez `prisma db push`
Uruchamianie	concurrently, nodemon	`npm run dev` uruchamia równolegle serwer (port 5000) i klienta (port 5173)

Szablonowe pliki Vite (przykładowe grafiki, `App.css`, README szablonu) zostały usunięte, a strona startowa wyświetla stan połączenia z API i bazą danych, co potwierdza poprawną integrację wszystkich warstw.

4. Standardy kodowania i ich automatyczne wymuszanie

4.1. Narzędzia

Narzędzie	Rola	Konfiguracja
Prettier 3	formatter: jednolity wygląd kodu	`.prettierrc.json`, `.prettierignore`
Oxlint 1	linter: wykrywanie błędów i złych praktyk w JS/JSX	jedna konfiguracja `.oxlintrc.json` dla klienta i serwera
EditorConfig	wspólne ustawienia edytora (wcięcia, kodowanie, LF)	`.editorconfig`
Husky + lint-staged	automatyczne uruchamianie lintera i formattera przed każdym commitem	`.husky/pre-commit`, sekcja `lint-staged` w `package.json`

Wybraliśmy Oxlint zamiast ESLint, ponieważ był już częścią szablonu Vite, działa kilkadziesiąt razy szybciej i nie wymaga instalowania wielu wtyczek. Reguły React, importów i dostępności (`jsx-a11y`) są w nim wbudowane.

4.2. Przyjęte standardy

- **Styl kodu (Prettier):** 2 spacje wcięcia, bez średników, pojedyncze cudzysłowy, przecinki na końcu wielolinijkowych obiektów i tablic, maksymalnie 100 znaków w linii, końce linii LF.
- **Reguły lintera:** kategoria *correctness* jako błąd (blokuje commit), *suspicious* jako ostrzeżenie; dodatkowo `eqeqeq`, `no-var`, `prefer-const`, `no-unused-vars`. W kliencie reguły hooków React (`rules-of-hooks`, `exhaustive-deps`) i dostępności.
- **Nazewnictwo:** komponenty React w `PascalCase` (`GoalCard.jsx`), hooki `useCamelCase`, pozostałe pliki, funkcje i zmienne w `camelCase`, endpointy jako rzeczowniki w liczbie mnogiej pod `/api`, modele Prisma w `PascalCase` w liczbie pojedynczej.
- **Git:** gałąź główna `main`, nowe funkcje na gałęziach `feature/<nazwa>`, poprawki na `fix/<nazwa>`, komunikaty commitów w trybie rozkazującym.

Pełny opis standardów znajduje się w sekcji „Standardy kodowania” w pliku `README.md`.

4.3. Skrypty i automatyzacja

Polecenie	Działanie
`npm run lint`	sprawdza cały kod linterem
`npm run lint:fix`	automatycznie poprawia to, co linter potrafi naprawić
`npm run format`	formatuje cały projekt Prettierem
`npm run format:check`	sprawdza formatowanie bez zmieniania plików
`npm run check`	lint i sprawdzenie formatowania razem

Po `npm install` Husky instaluje hook `pre-commit`. Przy każdym commicie `lint-staged` uruchamia na zmienionych plikach `oxlint --fix` i `prettier --write`, a commit z błędem lintera zostaje zatrzymany. Dzięki temu do repozytorium nie trafia kod niezgodny ze standardem.

```
"lint-staged": {
  "*.{js,jsx,mjs,cjs}": ["oxlint --fix", "prettier --write"],
  "*.{json,css,md,html,yml,yaml}": "prettier --write"
}
```

5. Prototyp interfejsu v2

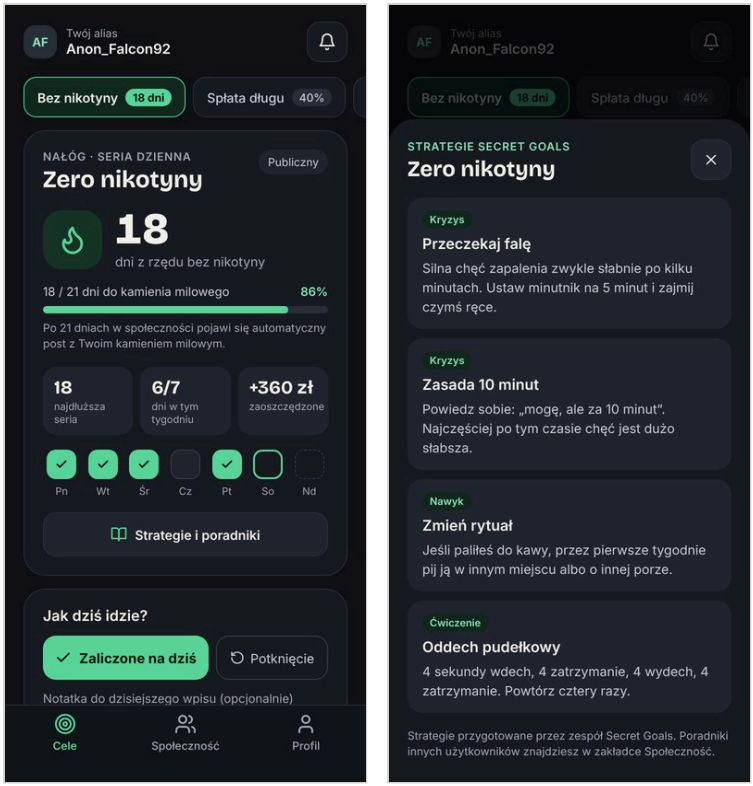
5.1. Założenia

Na podstawie wariantów A i B z Etapu 1 powstał nowy prototyp, który łączy ich najlepsze pomysły funkcjonalne, ale ma nowy wygląd. Aplikacja nie jest już ograniczona do PWA na smartfony: jest responsywną aplikacją webową z układem na telefon (nawigacja dolna) i na komputer (nawigacja boczna).

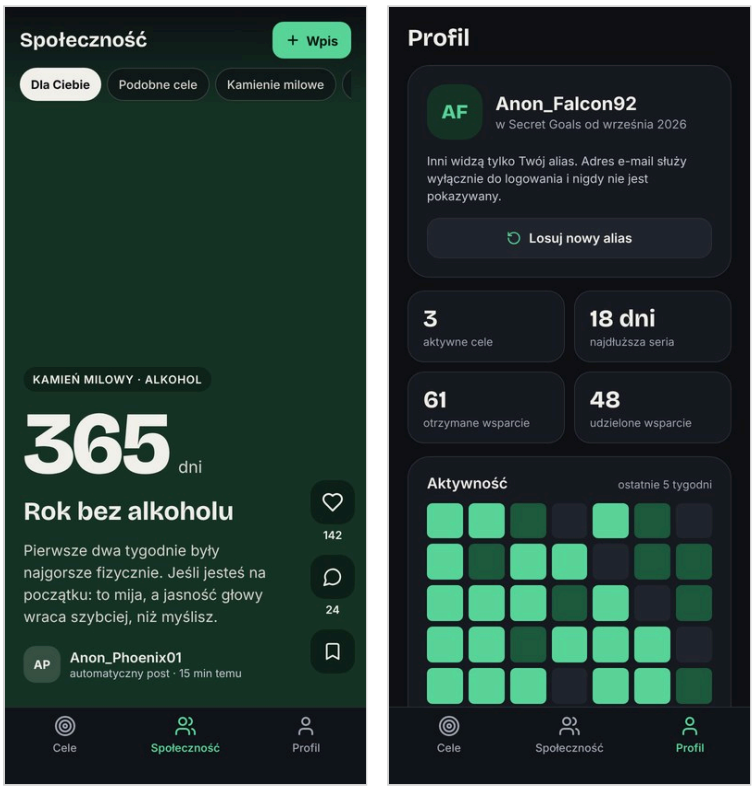
- **Cele:** u góry przełącznik między celami. W celu widać serię, pasek postępu do kamienia milowego, trzy małe statystyki i podgląd tygodnia. Postęp aktualizuje się przyciskami zależnymi od typu celu (seria dzienna: „Zaliczone na dziś” / „Potknięcie”, cel kwotowy: wpłaty, licznik tygodniowy: +1) oraz opcjonalnym meldunkiem publikowanym w społeczności.
- **Strategie i poradniki:** przycisk na dole każdego celu otwiera okno ze strategiami przygotowanymi przez zespół, dopasowanymi do kategorii celu.
- **Cel jako post:** publiczny cel jest automatycznie postem w społeczności, a odpowiednikiem polubienia jest „Wspieram”. Na ekranie celu widać liczbę wspierających, komentarze i osobę z podobnym celem (US05).
- **Społeczność:** pionowy feed pełnoekranowych wpisów w stylu rolek (meldunki, kamienie milowe, poradniki, nowe cele), przewijany scrollem na telefonie i komputerze, z filtrami: Dla Ciebie, Podobne cele, Kamienie milowe, Poradniki.
- **Profil:** anonimowy alias z możliwością wylosowania nowego, podsumowanie, mapa aktywności, widoczność celów i ustawienia.

Styl: ciemne tło, zielony akcent dla serii i głównych akcji, niebieski dla wsparcia, fonty Bricolage Grotesque i Inter. Interaktywny prototyp: claude.ai/artifact/UHzWL2uCYZhz1JscS3Xvne, źródła w `docs/prototypes/v2-ui/`.

5.2. Widoki na telefon

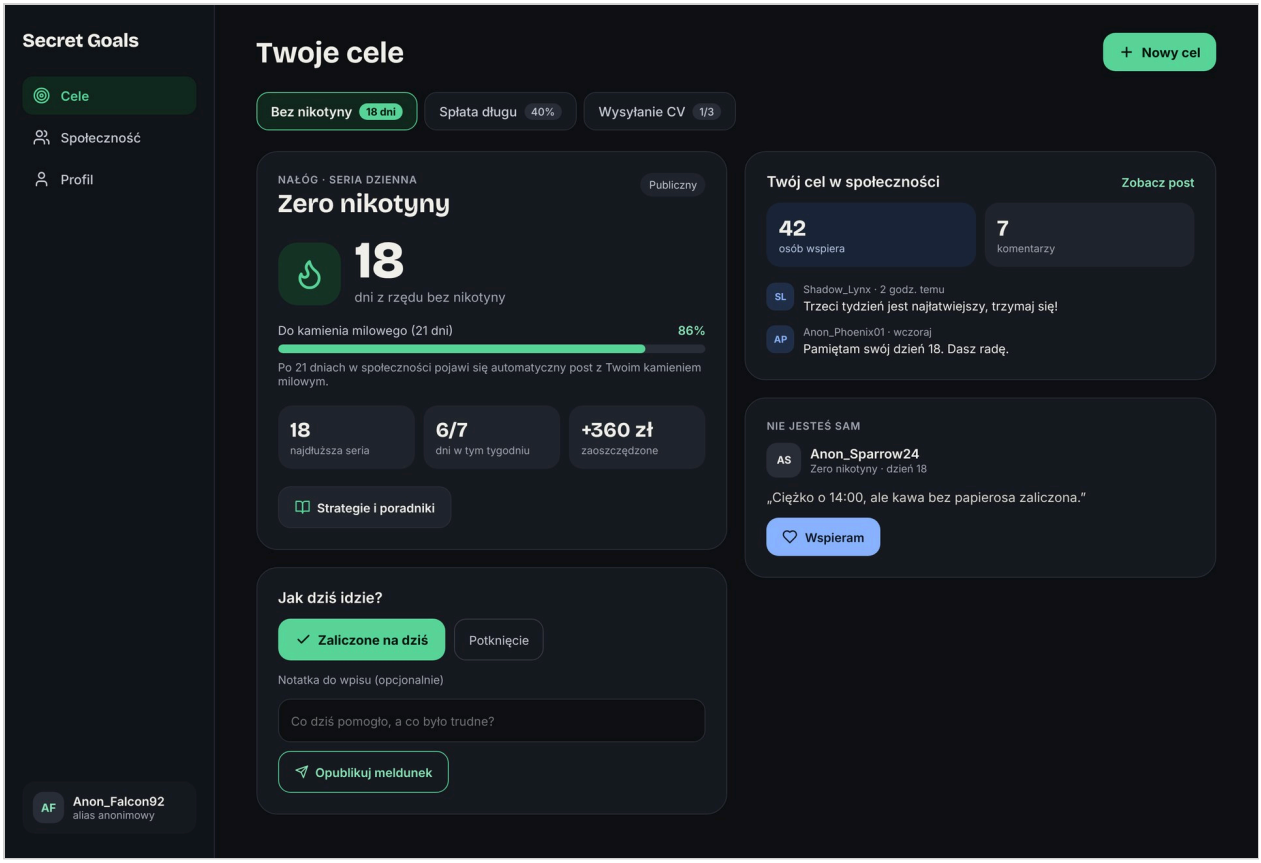


Rys. 1. Zakładka Cele oraz okno „Strategie i poradniki"

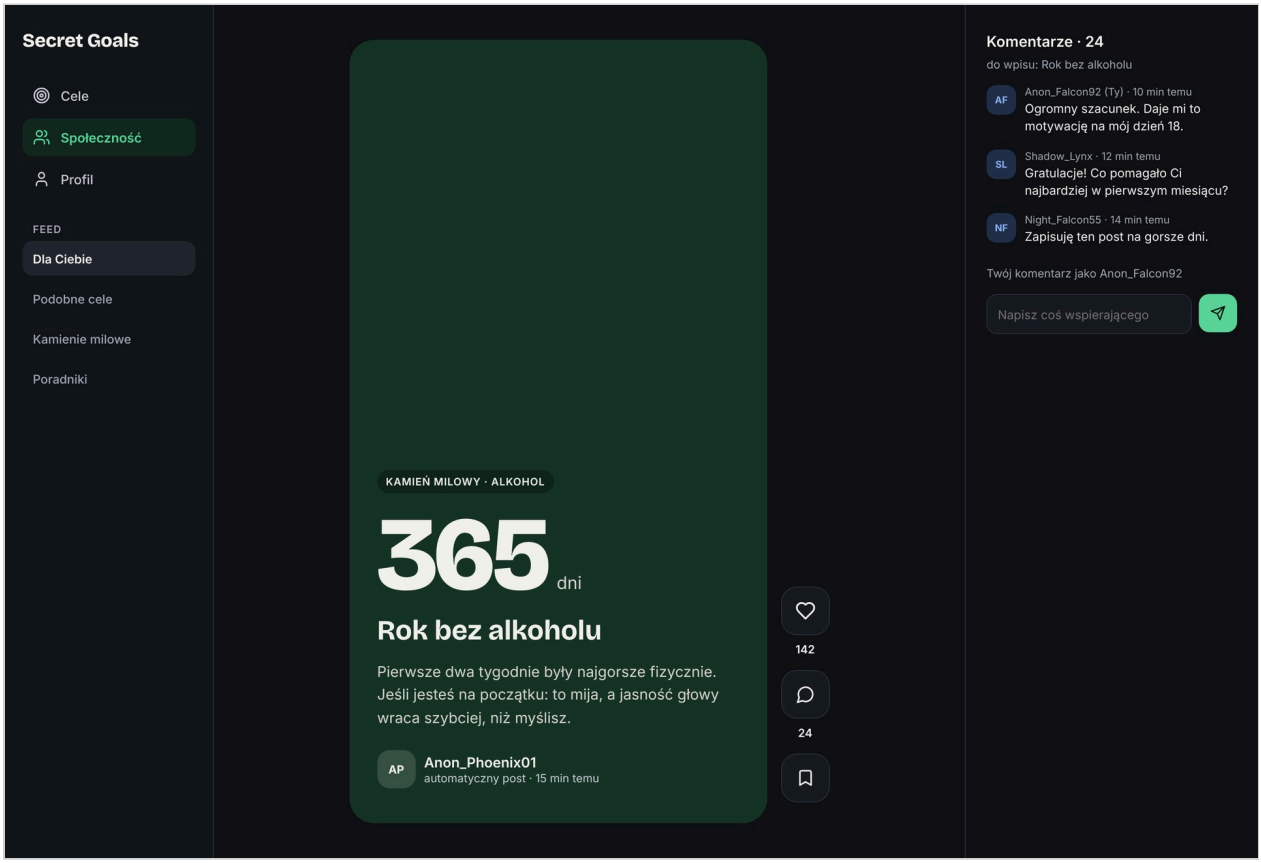


Rys. 2. Społeczność w formie rolek oraz Profil

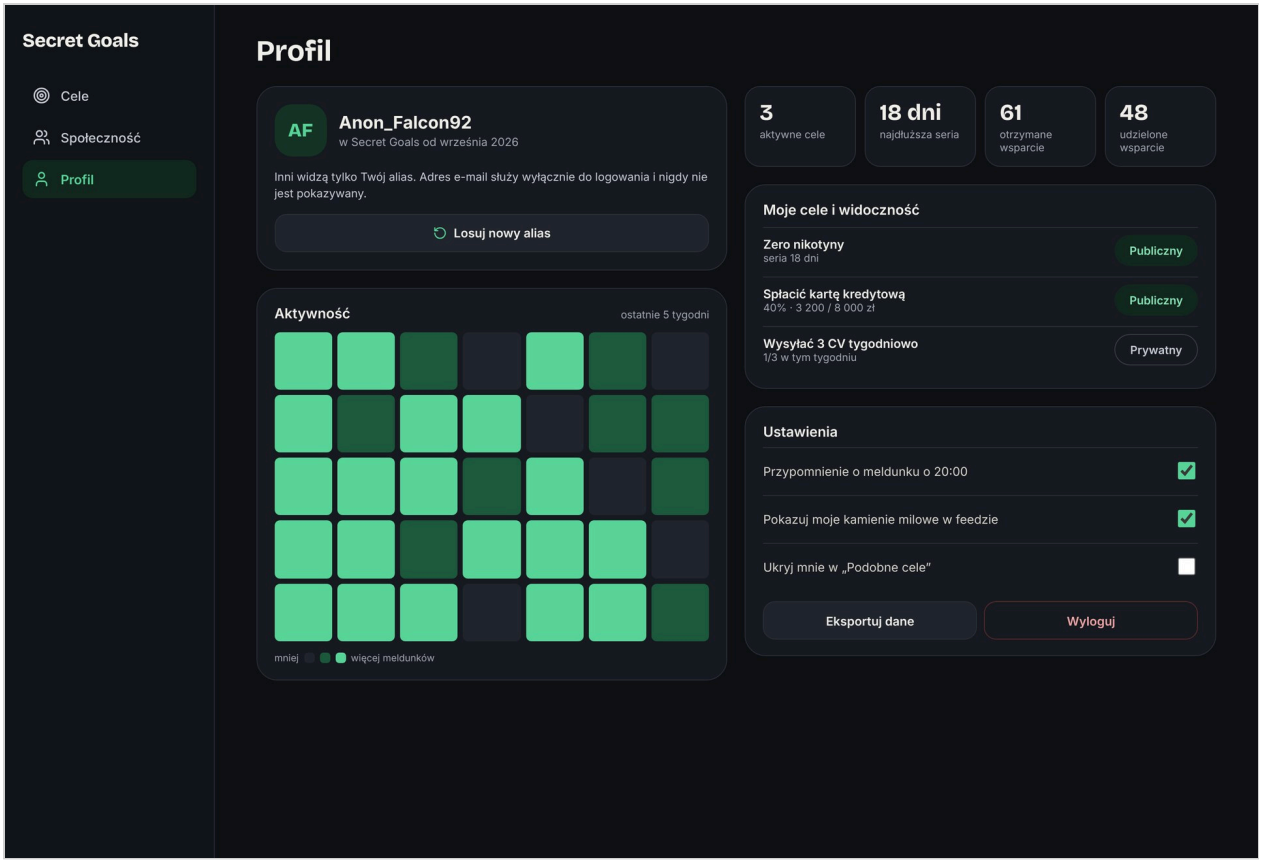
5.3. Widoki na komputer



Rys. 3. Zakładka Cele



Rys. 4. Społeczność: feed przewijany scrollem i komentarze aktualnego wpisu



Rys. 5. Profil

6. Podsumowanie realizacji wymagań

Wymaganie	Realizacja	Status
Repozytorium Git ze strukturą folderów zgodną ze standardem architektury	monorepo `client/`, `server/`, `docs/`; warstwy routes/controllers/services na serwerze, podział na pages/components/hooks/api w kliencie	**Spełnione**
Adekwatny plik .gitignore	wykluczone zależności, buildy, sekrety `.env`, lokalna baza SQLite i pliki edytorów; dodatkowo `.gitattributes`	**Spełnione**
Wybrany i zainicjowany framework	React + Vite + Tailwind, Express, Prisma + SQLite; działające endpointy i połączenie front–API–baza	**Spełnione**
Standardy kodowania	opisane w README: styl, reguły, nazewnictwo, konwencje Git	**Spełnione**
Automatyczne wymuszanie (linter, formatter)	Oxlint + Prettier, skrypty npm, hook pre-commit (Husky + lint-staged)	**Spełnione**

6.1. Kolejne kroki

- rozbudowa schematu bazy (postępy, posty, wsparcie, komentarze) i przejście z `db push` na migracje Prisma,
- rejestracja i logowanie z anonimowym aliasem (US01),
- implementacja zakładek Cele, Społeczność i Profil w React według prototypu v2,
- w dalszej perspektywie: konteneryzacja (Docker), CI/CD na GitHub Actions i wdrożenie na VPS.

7. Wykorzystanie narzędzi AI

Wykonawca	Obszar zastosowania	% objętości pracy
Claude + Gemini	Utworzenie nowej wersji prototypu interfejsu (v2), polepszenie szkieletu projektu: struktura folderów, konfiguracja lintera i formattera, hook pre-commit, dokumentacja standardów	70%
Łukasz Michno	Myślenie koncepcyjne i założenia funkcjonalne, podejmowanie decyzji projektowych, analiza wymagań i wyników, testowanie	30%